A Hyper-paramerer settings

All hyper-parameter settings for Section 3 are listed in Table A1.

Table A1: Hyper-parameter settings

Hyper-parameter	Description	Value
α	Balance coefficient between local and global	0.9
c	Exploration coefficient in UCT function	1.4
γ	Discount factor for backpropagation	0.1
max_child	Dynamic pruning hyper-parameter in search	2

B Details of Grading and Metrics

Grading. We follow the correctness evaluation method proposed in PRM800K (Lightman et al., 2023) to score the answers generated by the algorithm. For the mathematical solutions proposed by the algorithm and their corresponding ground truth, we inform the model of the expected response format in a prompt. The answer's formula or value is extracted by matching the response format with predefined rules. If the model fails to follow the expected format in the prompt and the rule-based extraction fails, the solution is directly judged as inconsistent with ground truth.

For the extracted label, we score the answer based on the following criteria. The answer is considered consistent with the ground truth label and passes the evaluation if at least one of the criteria is met:

1. The answer label string is exactly equal to the ground truth label string in terms of literal value.

2. Both the answer label and the ground truth label can be converted to floating-point numbers, and the difference between the two values is less than 1×10^{-6}.

3. Following the criterion proposed in PRM800K (Lightman et al., 2023), we use the Sympy library (Meurer et al., 2017) to simplify the difference between the expression corresponding to the answer label, denoted as a, and the expression corresponding to the ground truth label, denoted as b. If the simplification yields $a - b \iff 0$, the criterion is satisfied.

Metrics. To provide a comprehensive and robust evaluation metric, we adopt **major@k** and **rm@k** as the evaluation metrics.

- **rm@k** represents reward model best-of-N among k sampled response (Yang et al., 2024c).

- **major@k** (Kuncheva, 2014), also abbreviated as **maj@k**, is defined as the fraction of tasks where a majority of the top k samples generated return the same correct solution. This metric focuses on consistency across multiple generated answers. The majority weight is calculated on the solution's extracted labels.

- The **solved rate of problems** (Lightman et al., 2023; Luo et al., 2024a) refers to the percentage of problems who have solutions meet the evaluation criteria.

For results without a subscript, the score represents the greedy evaluation using the default prompt of the base model. Results with a notation indicate the use of the corresponding prompt engineering technique, and those marked with **self-consistency** use the self-consistency aggregation method.

For closed-source models, we report the scores from existing official technical reports (Anthropic, 2024; Reid et al., 2024; OpenAI, 2024) or dataset-provided results, with no modifications.

C Details of PPRM Trainging

C.1 Overview

The design goal of the PPRM is to integrate the properties of both PRM and ORM, providing a more nuanced preference prediction between any two solution answers. We attempt to utilize reinforcement learning methods for training, leveraging the model's instruction-following capability to predict the relative merits of pairs of problem-solving answers. This will further enable the use of the EBC method to evaluate the global quantile scores of mathematical solutions.

C.2 Data Collection

Our data synthesis is derived from two datasets: PRM800K (Lightman et al., 2023) and OpenMathInstruct-1 (Toshniwal et al., 2024). The PRM800K dataset, collected from the MATH dataset, comprises a substantial number of step-divided problem-solving answers, with manual quality annotations for each step. We primarily utilize this dataset to generate pairs of answers for comparative analysis based on step-wise process quality. The OpenMathInstruct-1 dataset

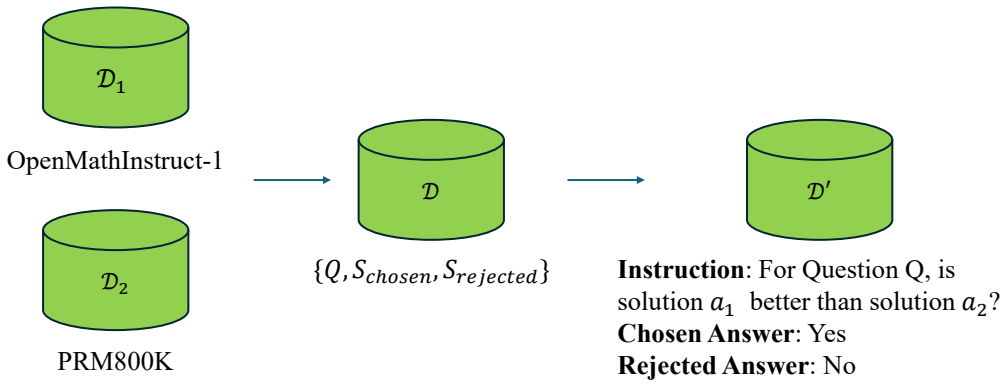

Figure A1: Dataset Construction of PPRM.

incorporates data from the GSM8K and MATH datasets, which have been manually annotated for outcome correctness. We use this dataset to synthesize pairs for comparative analysis based on outcome quality.

In processing PRM800K, for a given problem, we first sample steps of varying quality annotations from the step-wise dataset to construct a complete reasoning path. For pairs with the same final step annotation, paths composed of higher-quality steps are considered superior to those with lower-quality steps. In cases where the final step annotations are not identical, the reasoning path with the superior final step annotation is regarded as better.

During the processing of OpenMathInstruct-1, we exclusively utilize samples without Python interpreter calls. For the same problem, we sample pairs composed of outcomes with higher and lower quality annotations.

Notably, we filtered out any data samples from PRM800K and OpenMathInstruct-1 that may overlap with the GSM8K and MATH test sets, especially the PRM800K. Ultimately, we formed a dataset of 7,780,951 entries for training the PPRM model.

C.3 Direct Preference Pair Construction

For all pairs, we frame the inquiry as "For Question Q, is solution a_1 better than solution a_2?" If solution a_1 is deemed superior to solution a_2, we label it as 'Yes'; otherwise, it is labeled as 'No'.

In this manner, we transform the ordinal relationship prediction task into a question-answer format, employing the Direct Preference Optimization (DPO) method for model training through reinforcement learning from human feedback (RLHF). This approach aims to enhance the model's capability to follow instructions in predicting the relative merits of pairs of problem-solving answers.

C.4 RLHF Training

We apply the DPO method to train the Gemma2-2B-it model using RLHF which is shown in Figure A2. The loss function for DPO is structured as $\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$, where σ is the logistic function, β is a parameter controlling the deviation from the base reference policy π_{ref}, namely the initial model π^{LLM}. In practice, the language model policy π_θ is also initialized to π^{LLM}. For one answer, denoted as $y_w \succ y_l|x$ where y_w and y_l denote the preferred and dispreferred completion amongst $(\hat{y}, y)$ respectively.

D Details of Berry-Tree Inference Framework

D.1 Overview

`Berry-Tree` is an inference framework specifically designed for complex multi-model reasoning tasks, addressing the need to improve inference efficiency and accuracy in intricate tree search processes of LLM's mathematical reasoning. This framework is particularly suited for large-scale reasoning tasks that involve the integration of multiple models and algorithms. By incorporating advanced tree search methods especially Monte Carlo Tree Search (MCTS), robust concurrency handling, and high-performance inference engines, `Berry-Tree` significantly enhances inference speed and resource utilization of LLM's mathematical reasoning process. This section provides a explanation of the system architecture and key tech-

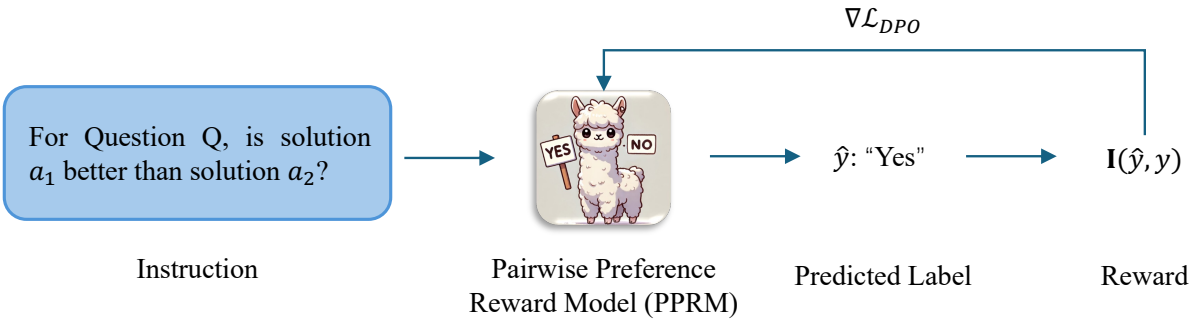

Figure A2: RLHF training of PPRM.

nologies of `Berry-Tree`, along with a preliminary performance evaluation results.

D.2 System Architecture Overview

Figure A3 demonstrates the architecture of `Berry-Tree` which is divided into several layers, each handling different functional requirements. The **Data Management Layer** is responsible for the serialization and deserialization of data, ensuring efficient data read/write operations across models and systems and the ablity of recovering the search process from serialized data. The **Tree Search Methods Layer** incorporates MCTS (Monte Carlo Tree Search), ToT (Tree of Thoughts), and A* algorithms to optimize the inference process and explore multiple reasoning paths. Additionally, `Berry-Tree` includes a **Reliability Layer**, which ensures load balancing and failover support in highly concurrent scenarios, guaranteeing the stability of inference tasks. Finally, the **Inference Engines Layer** integrates efficient inference engines such as vLLM, FastAPI, and HuggingFace TGI to enable parallelized and efficient task processing.

D.3 Key Technical Components

Data Management. `Berry-Tree` employs serialization and deserialization techniques, specifically using formats including JSON and CSV, to efficiently store, transfer and recover checkpoint data from tree search reasoning processes. The framework stores this data along with hash values to ensure integrity and allows for quick restoration of tree search states in memory when needed. Furthermore, `Berry-Tree` leverages the HuggingFace Datasets library to handle core dataset inputs for both training and inference. It supports seamless loading of benchmark datasets such as GSM8K and MATH from the HuggingFace Hub, enhancing its flexibility and ensuring compatibility with diverse reasoning tasks.

Tree Search Methods. `Berry-Tree` support multiple tree search algorithms, with MCTS being central to handling large-scale complex reasoning tasks by leveraging random simulation and statistical analysis to optimize the search space. The Tree of Thoughts (ToT) extends the exploration depth and breadth of reasoning paths, helping the system manage uncertainty. And A* can offer a efficient heuristic search capabilities.

Reliability Design. To ensure the stability and continuity of inference tasks, `Berry-Tree` incorporates load balancing and failover mechanisms. During high-concurrency operations, the load balance componet distributes workloads across different inference servers, preventing server overload. The failover mechanism ensures that tasks can seamlessly recover and transition to backup servers in case of partial server failures.

Server Architecture. The framework's server architecture is divided into two segments: one dedicated to executing Large Language Model (LLM) inference and the other designated explicitly for handling PPRM (Pairwise Preference Reward Model) with EBC (Enhanced Borda Count) method. This modular design allows the framework to allocate computational resources flexibly, improving overall efficiency.

Inference Engines. Inference engines of `Berry-Tree` include VLLM , HuggingFace Transformers warpped by FastAPI, and HuggingFace TGI. These engines collectively enable the system to maintain high efficiency and stability while handling multi-model inference tasks, with robust support for high-concurrency demands.